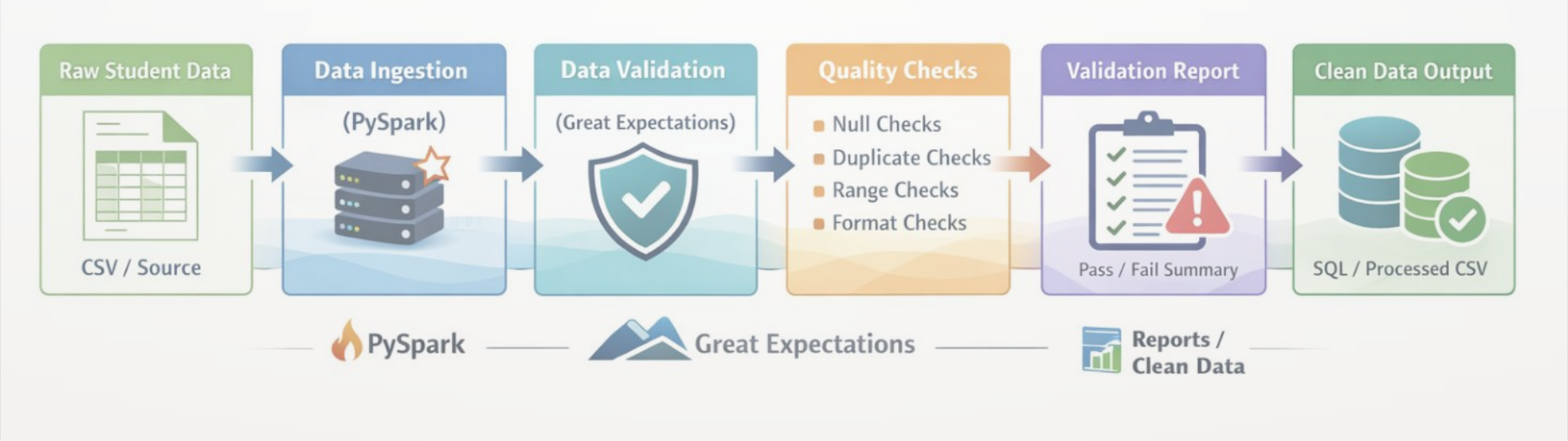
Data Quality Framework using PySpark

Project Presentation



TEAM- 2

Esha-2320030381

Charvi-2320030371

Rushita-2320030064

KL ACCREDITED BY NAAC WITH A++ GRADE

nirf 2022 NATIONAL INSTITUTIONAL RANKING FRAMEWORK RANKED 27 AMONG ALL UNIVERSITIES

CATEGORY 1 UNIVERSITY BY MHRD, Govt. of India

43 YEARS OF EDUCATIONAL LEADERSHIP

CONTENTS

- Abstract
- Introduction
- Literature Survey
- Limitations and Proposed Solutions
- Methodology / Objectives
- Implementation
- Testing
- Deployment

ABSTRACT

Data quality plays a vital role in ensuring accurate analysis, reliable reporting, and effective decision-making in modern data-driven systems. However, real-world datasets often contain various data quality issues such as missing values, duplicate records, invalid formats, and inconsistent or out-of-range values. These issues can negatively impact analytics results, reduce system reliability, and lead to incorrect business insights. Therefore, it is essential to implement mechanisms that can automatically detect and manage data quality problems before data is used for further processing.

This project presents the design and implementation of an automated Data Quality Framework that validates datasets and ensures data integrity. The framework is developed using Python, PySpark, and Great Expectations, which are widely used tools in big data and data engineering environments. It ingests raw data, applies a set of predefined validation rules such as null checks, uniqueness constraints, range validation, and format verification, and identifies records that do not meet quality standards. The framework also generates validation reports that provide detailed information about data quality issues.

The system follows a modular and scalable approach, making it suitable for integration into real-world data pipelines. By automating the data validation process, the framework reduces manual effort, improves efficiency, and ensures that only clean and reliable data is used for analytics and decision-making. This project demonstrates key data engineering concepts such as data ingestion, validation, and reporting, and highlights the importance of maintaining high data quality in building robust and trustworthy data systems.

INTRODUCTION

In modern data-driven systems, the accuracy, completeness, and reliability of data are critical for making correct business decisions and performing meaningful analytics. Poor data quality such as missing values, duplicate records, incorrect formats, or invalid ranges can lead to inaccurate insights, system failures, and loss of trust in data systems. Therefore, ensuring high data quality is an essential responsibility of data engineers.

This project focuses on designing and implementing a Data Quality Framework that automatically validates datasets before they are used for analysis or downstream processing. The framework performs a series of predefined validation checks, such as null value detection, uniqueness constraints, range validation, and format verification, to identify and report data quality issues.

The framework is built using Python, PySpark, and Great Expectations, which are widely used tools in modern data engineering workflows. It ingests raw data, applies data quality rules, generates validation reports, and helps ensure that only clean and reliable data proceeds further in the data pipeline.

This project demonstrates key data engineering concepts, including data ingestion, validation, transformation, and reporting. By implementing this framework, the project highlights the importance of maintaining data integrity and provides a scalable solution that can be integrated into real-world data pipelines to improve data reliability and trustworthiness.

Existing System

In many organizations, data is collected from multiple sources such as databases, logs, APIs, and user inputs. However, these datasets often contain inconsistencies such as duplicate records, missing values, incorrect formats, and invalid ranges. The existing systems usually store and process this data without performing proper validation or quality checks.

Due to the absence of automated data validation mechanisms, poor-quality data enters analytics pipelines and decision-making systems. This results in inaccurate analysis, unreliable insights, and operational inefficiencies. Manual data validation is often time-consuming and prone to human error, making it difficult to maintain data integrity at scale.

Therefore, traditional systems lack automated frameworks to continuously monitor and validate the quality of incoming data.

Limitations of Existing Frameworks and our proposed Solutions

Existing Drawbacks	How our project overcame it
Limited to intrinsic quality only	Add multiple quality dimensions; completeness, accuracy, consistency, validity, timeliness
No real-time validation	Add streaming validation using Spark Structured Streaming
Static validation rules	Implement dynamic rule engine using config file(JSON/YAML)
Limited scalability	Use PySpark distributed processing
No automated reporting	Generate automated quality reports(CSV/HTML)
Domain-specific frameworks	Build domain-independent engine
No software implementation	Full working implementation with modular code
Cannot detect integrated data errors	Add cross-column and cross-table validation
Manual quality checks	Fully automated pipeline

PROPOSED SYSTEM

- The proposed system introduces an automated **Data Quality Framework** that ensures the integrity, consistency, and accuracy of data before it is used for analysis. The system uses **PySpark** for scalable data processing and **Great Expectations** for implementing data validation rules.
- The framework first ingests raw data and processes it using Spark. Validation rules are then applied to detect issues such as duplicate records, null values, invalid ranges, and incorrect formats. After identifying these issues, the system cleans the dataset by removing or correcting invalid entries.
- Finally, the cleaned dataset is revalidated to ensure that it meets all quality standards. This automated pipeline improves data reliability and ensures that only high-quality data is used for further analysis.

MODULES & OBJECTIVES

Objectives

- Ensure high data quality in big data pipelines
- Automatically detect data quality issues
- Build scalable validation using PySpark
- Generate automated validation reports

System Modules

- Data Ingestion Module
- Data Validation Module
- Rule Engine Module
- Reporting Module
- Clean Data Output Module

LITERATURE SURVEY

LINK	JOURNAL	DRAWBACKS	FUTURE DIRECTIONS
https://pmc.ncbi.nlm.nih.gov/articles/PMC8019177/	BMC Medical Research Methodology	Limited to intrinsic data quality; excludes task-specific or contextual aspects like currency in EHRs; relies on metadata availability.	Extend to other study types and data quality aspects; integrate more process variables for broader applications.
https://pmc.ncbi.nlm.nih.gov/articles/PMC6568139/	AMIA Joint Summits on Translational	Requires predefined domain ontologies, which demand significant upfront work; does not assess all quality aspects like timeliness or correctness fully; tested on moderate-sized datasets only.	Make HDQF open-source; test on larger national datasets across organizations; develop additional measures for timeliness and correctness.
https://arxiv.org/abs/2403.00526	Science Proceedings arXiv preprint	Systematic assessment of dimensions like accuracy is largely absent; challenges in gauging cleaning success; limited to proposed facets without full empirical validation.	Develop comprehensive framework overcoming data/source/system/task/human facets; align with regulations like EU AI Act.
https://pmc.ncbi.nlm.nih.gov/articles/PMC11297942/	Not specified (PMC)	Focused on ML/DL in medicine; adoption challenges in high-stakes areas due to unproven trustworthiness.	Broaden trustworthy AI development for medical ML applications.
https://pmc.ncbi.nlm.nih.gov/articles/PMC5977591/	Not specified (PMC)	Modified existing framework for specific domain (heart failure); may not generalize without re-operationalization.	Apply to broader clinical domains beyond heart failure.

LITERATURE SURVEY

LINK	JOURNAL	DRAWBACKS	FUTURE DIRECTIONS
https://pmc.ncbi.nlm.nih.gov/articles/PMC10646672/	Not specified (PMC)	Limited to digital health tech interventions; gaps in evaluating real-world health research conduct.	Identify more interventions for data-based health research
https://unstats.un.org/unsd/ccsa/cdqio-2010/Ses1-DQAF-IMF.pdf	IMF Document	Covers prerequisites and five dimensions but generic; lacks software implementations for modern use.	Update for current statistical production needs.
https://nces.ed.gov/fcsm/pdf/FCSM.20.04_A_Framework_for_Data_Quality.pd	FCSM Report	Identifies threats but needs methods for cumulative errors in integrated data; limited disclosure risk assessment.	Develop methods for integrated data errors, gold standards, and disclosure risk
https://gsconlinepress.com/journals/gscarr/content/data-quality-metrics-and-frameworks-comprehensive-study-ensuring-reliable-inf	GSCARR	Persistent challenges in big data accuracy/completeness despite growth; framework alignment with real-time analytics.	Propose integrated DQM for big data, warehousing, real-time analytics
https://medinform.jmir.org/2025/1/e75275	JMIR Medical Informatics	Static evaluations don't adapt to changing data pipelines; limited to core dimensions like completeness.	Advance AIDAVA for dynamic health data integration.

SOFTWARE REQUIREMENTS

The following software tools and technologies are required to implement the system:

- **Python** – Used for implementing the data processing logic.
- **PySpark** – Used for large-scale distributed data processing.
- **Great Expectations** – Used for defining and executing data validation rules.
- **Google Colab / Jupyter Notebook** – Used as the development environment.
- **Pandas** – Used for dataset manipulation and cleaning.
- **Matplotlib** – Used for visualizing data distributions.
- **Apache Spark** – Provides the distributed processing engine.

HARDWARE REQUIREMENTS

The system requires the following hardware components:

- **Processor:** Intel i3 or higher
- **RAM:** Minimum 8 GB
- **Storage:** Minimum 20 GB free space
- **Operating System:** Windows / Linux / macOS
- **Internet Connection:** Required for installing libraries and running cloud-based notebooks.

These requirements ensure smooth execution of data processing and validation tasks.

FUNCTIONAL REQUIREMENTS

The functional requirements of the system include:

- Loading and processing raw datasets.
- Detecting duplicate records.
- Checking for null or missing values in important fields.
- Validating numerical ranges such as marks and attendance.
- Validating email formats using regular expressions.
- Cleaning invalid or corrupted data records.
- Generating validation results and statistics.
- Saving the cleaned dataset for further analysis.

NON-FUNCTIONAL REQUIREMENTS

Non-functional requirements define the quality and performance characteristics of the system :

- **Scalability:** The system should handle large datasets efficiently using PySpark.
- **Performance:** Data validation should be performed quickly even for large datasets.
- **Reliability:** The framework should consistently detect and report data quality issues.
- **Maintainability:** Validation rules should be easy to modify or extend.
- **Usability:** The system should be easy to execute using a simple notebook interface.

SYSTEM ARCHITECTURE

The system architecture consists of the following components:

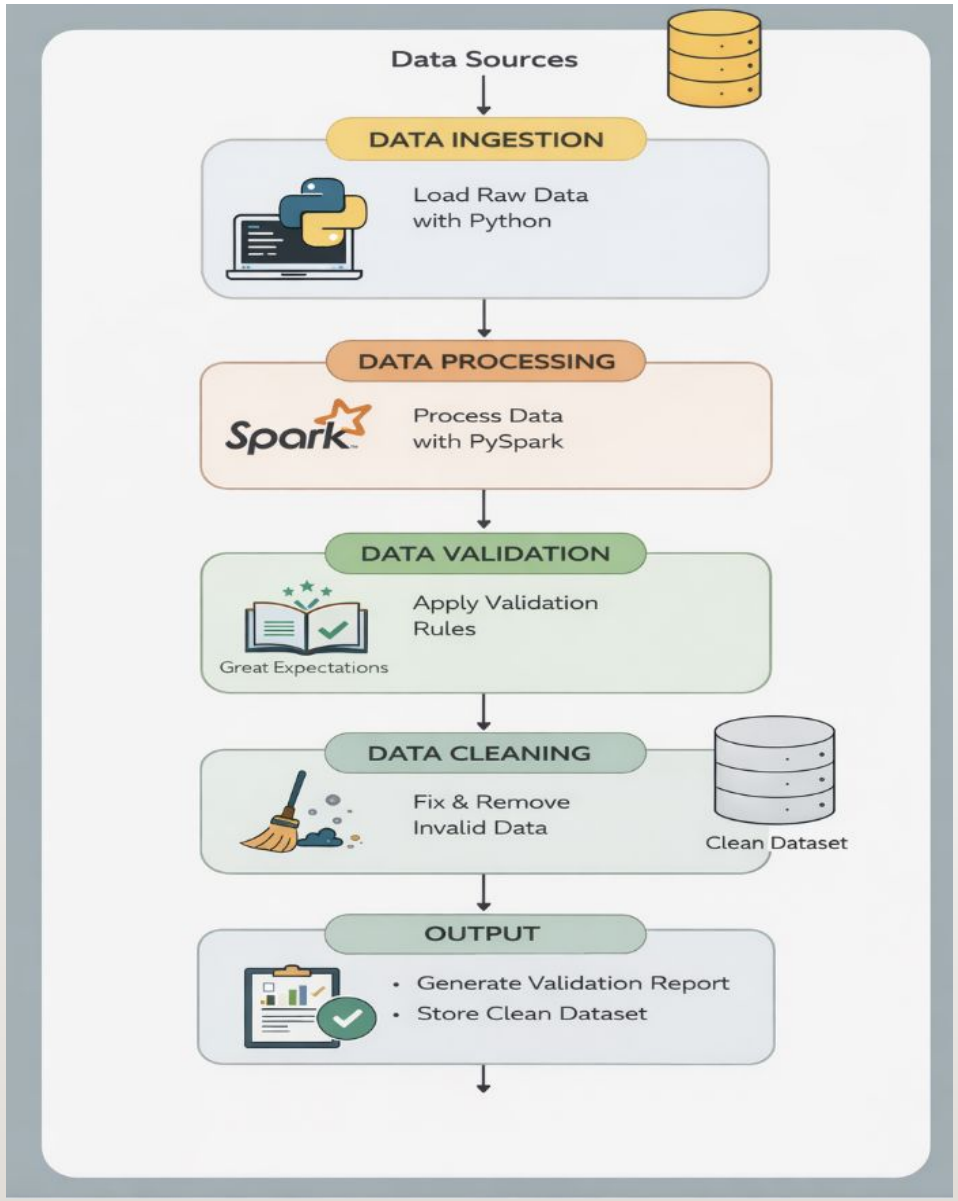
1. **Data Ingestion Layer**
   Raw datasets are loaded into the system using Python and Spark.

2. **Data Processing Layer**
   PySpark processes the dataset and prepares it for validation.

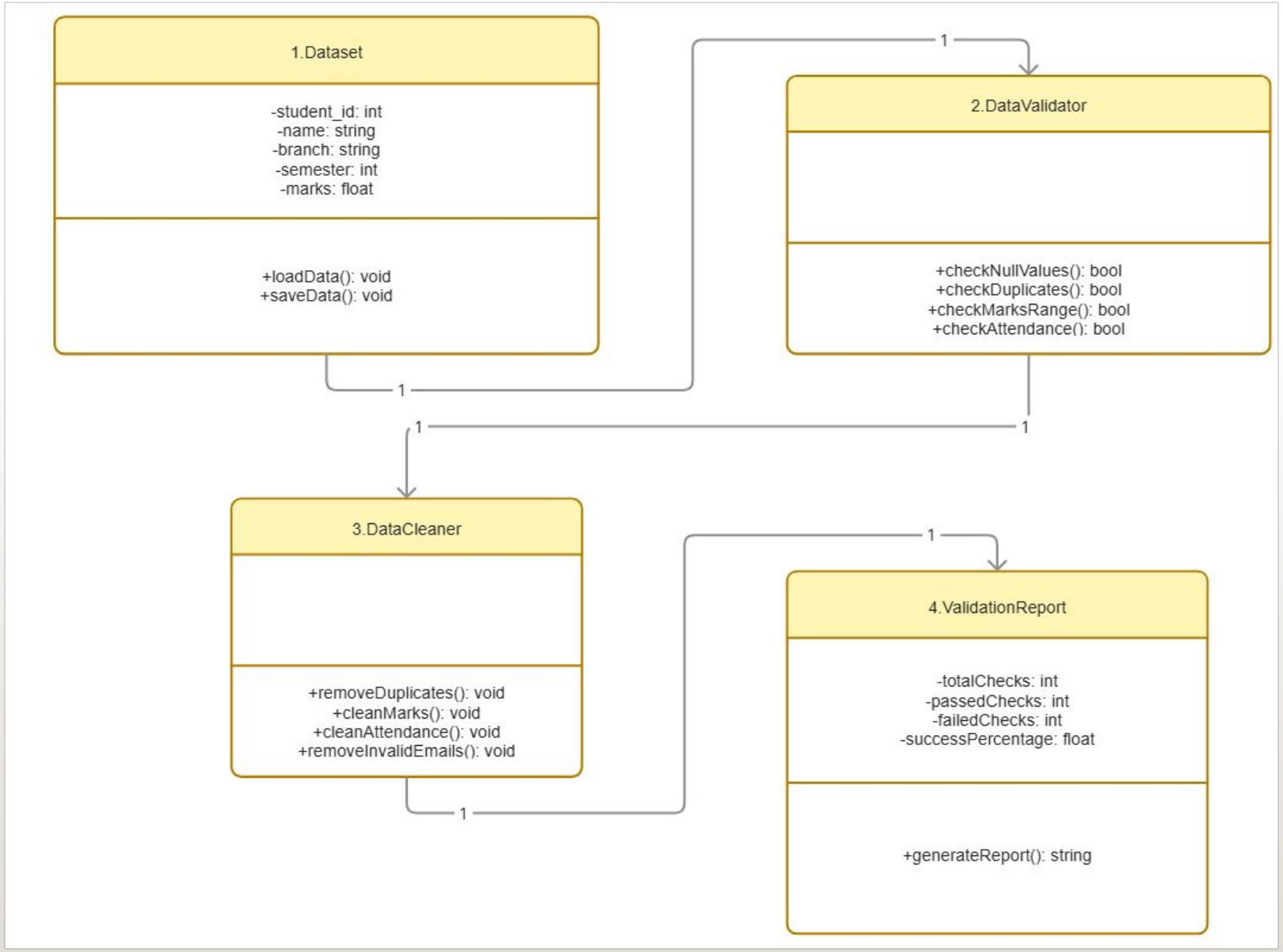
3. **Data Validation Layer**
   Great Expectations applies predefined validation rules such as checking for duplicates, null values, range limits, and format validation.

4. **Data Cleaning Layer**
   Invalid records are removed or corrected based on validation results.

5. **Output Layer**
   The cleaned dataset is stored and made available for analysis or visualization.



SYSTEM ARCHITECTURE


1.Dataset
-student_id: int
-name: string
-branch: string
-semester: int
-marks: float
+loadData(): void
+saveData(): void
2.DataValidator
+checkNullValues(): bool
+checkDuplicates(): bool
+checkMarksRange(): bool
+checkAttendance(): bool
3.DataCleaner
+removeDuplicates(): void
+cleanMarks(): void
+cleanAttendance(): void
+removeInvalidEmails(): void
4.ValidationReport
-totalChecks: int
-passedChecks: int
-failedChecks: int
-successPercentage: float
+generateReport(): string
1
1
1
1
1
1

UML DIAGRAM

THANK YOU

Team 2